

# AXONA

## A programmer's introduction

*Self-authenticating, geo-aware pub/sub that runs in a browser tab — with no platform in the middle.*

June 2026 · v0.4 · a ~30-minute talk

Live: [demo.axona.net/apps/axona-minimal](https://demo.axona.net/apps/axona-minimal) · Docs: [github.com/axona-net/axona-docs](https://github.com/axona-net/axona-docs)

You'll leave with a working mental model of the network, the small API you actually call, and an app you watched run live — **Axona Minimal**, about sixty lines, publishing and subscribing over the real mesh.

# What Axona is

- **A peer-to-peer pub/sub bus.**

Any peer publishes to a topic; any peer subscribes; messages flow peer-to-peer over an encrypted WebRTC mesh. No broker in the path.

- **Self-authenticating.**

A peer's identity *is* its keypair, and its address is derived from its public key and verified directly. No account, no certificate authority, no registrar.

- **It runs in a browser tab.**

Pure JS/WASM; the same kernel runs in Node and in the simulator. The cost of entry is a keypair and a page.

## THE SHIFT

A hosted broker — Pusher, Ably, a Kafka cluster — is a company you trust and a switch someone can flip.

Axona moves that function *into the participants*. Nothing to sign up for, nothing to revoke through, and no chokepoint to censor or seize.

## LIVE NOW

The first version is in the wild; anyone can connect.

# Addresses are places

- A node id is 264 bits.

Top 8 = your S2 geographic cell, the rest = SHA-256 of your key. XOR distance on that top byte makes routing favor locality.

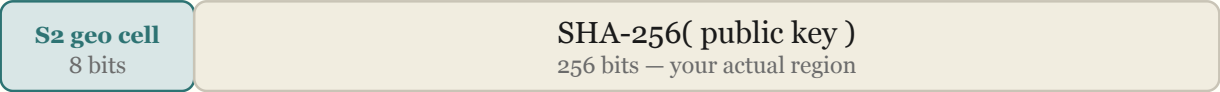
- A topic id, same shape — you pick the region.

The S2 prefix is the publisher's choice, not your own cell, so a topic roots anywhere (the demo: us-east).

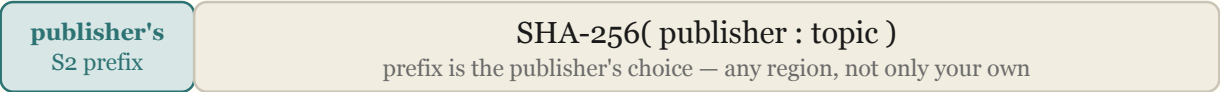
- The publish id carries no geography.

Just the signing key — no S2 prefix — so a post proves *who* wrote it, never *where* they are.

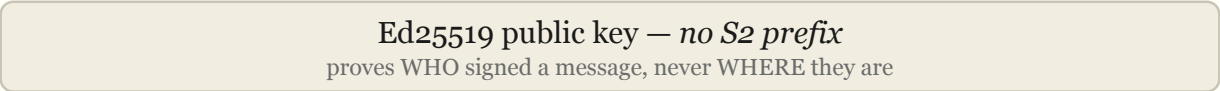
**node id** — 264 bits · your place in the mesh



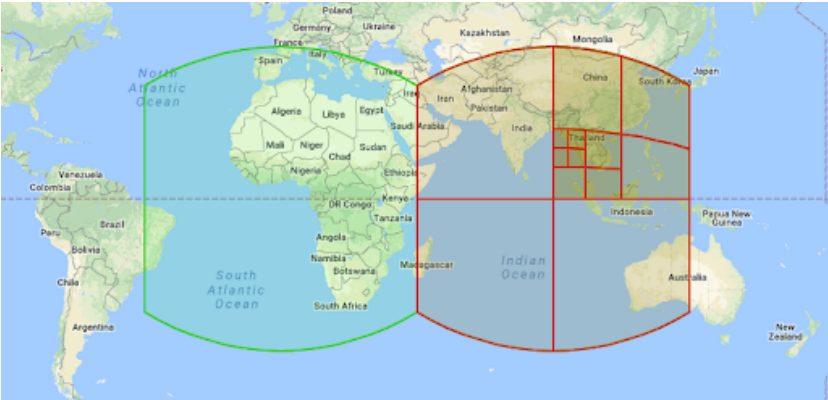
**topic id** — same shape, but you pick the region



**publish id** — the key that signs a post · *no geography*



192 CELLS COVER THE PLANET



A node roots the topics near it; a sparse region's root set simply spills to the nearest neighbors. Geography is a routing hint, not a border.

# The mesh, and how a message travels

- A bridge bootstraps; the mesh carries.

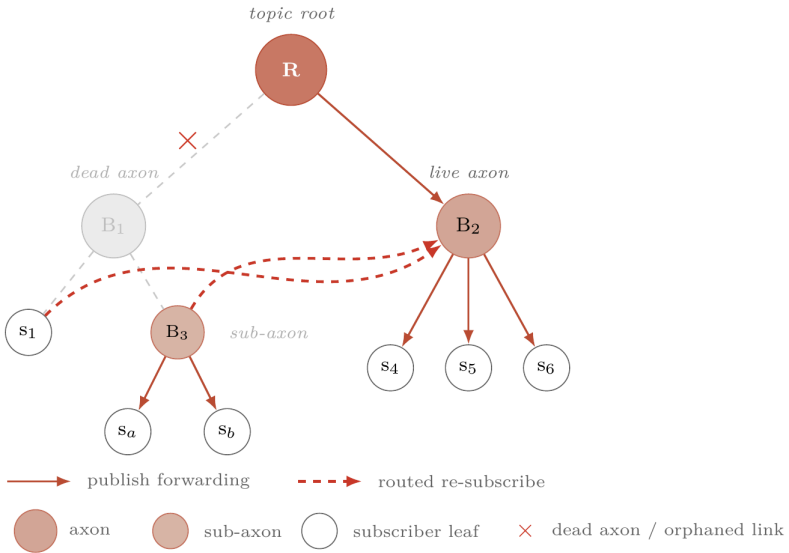
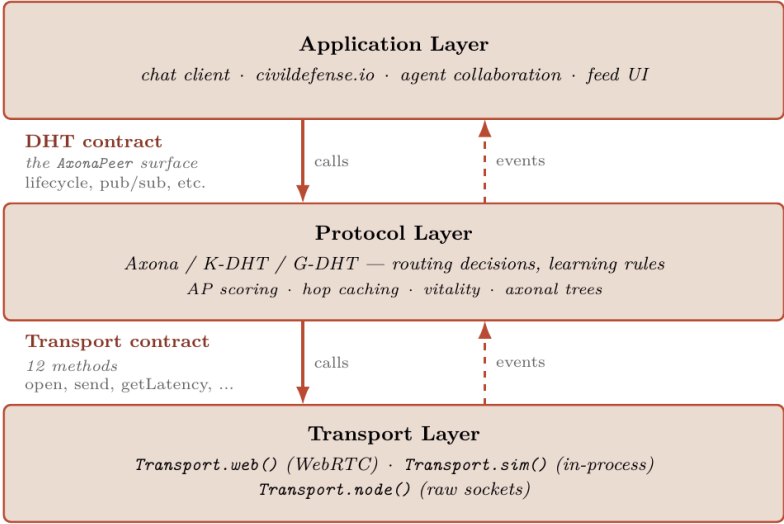
A WebSocket bridge introduces peers and relays signaling — a rendezvous, not a router. Peers form a WebRTC mesh and reach any id in a few hops via a learned routing table.

- Topics have a root set.

The R nodes closest to a topic id replicate it. A publish lands on the roots and fans out down a tree to the subscribers.

- It heals.

Gap-safe replay, root-to-root anti-entropy, and kill convergence keep replicas consistent — a missed message is backfilled, a retraction stays retracted.



# The security model

- **Identity = keypair.**

The address is derived from the public key, and the transport handshake is mutually authenticated and channel-bound — a man-in-the-middle that swaps the media fingerprint fails the bind.

- **Signed, verified at ingress.**

A signed publish is checked against the publisher's key at the root, before it is cached or fanned out. Spoofed-signature spam dies at the edge.

- **You can only act for yourself.**

A subscribe or unsubscribe is honored only for the authenticated peer's own id — no subscribing a victim, no silencing one.

- **Who, not where.**

A signed post proves its author by key, but the envelope carries no location — the publisher's S2 region is never on the wire. An app reveals a sender's region only by choosing to.

- **Bounded & content-addressed.**

A node only roots topics it is closest to; the message id is the hash of its content; a per-publisher sequence and TTL drop stale or replayed envelopes. Memory-hard proof-of-work prices a publish identity against Sybils.

## NO TRUST SERVER

Every guarantee here is carried by the peers' own cryptography. No CA, no central authority sits behind any of it.

## WHAT IT DOES *NOT* PROMISE

A kill is best-effort redaction, not a cryptographic un-send: a peer that already holds the bytes keeps them. And a medium no platform can censor is one no platform can moderate. Design with that in mind.

# The API you actually call

- **Construct a peer.**

```
deriveIdentity() → webTransport() → new AxonaPeer() → peer.start().
```

- **Then publish and subscribe.**

```
peer.pub(topic, msg, {publisher}) and peer.sub(topic, handler, {publisher, since}).
```

That is the whole core loop.

- **The rest is on demand.**

```
unsub, pull (by id, or latest), kill (creator-only retract), host (serve a topic without consuming it), metrics, health.
```

WHERE A TOPIC LIVES

opts.publisher selects it:

| value   | the topic is... |
|---------|-----------------|
| omitted | your own feed   |
| null    | a public topic  |
| hex id  | someone's feed  |

A shared room uses a region-anchored publisher, so every peer in the region derives the same topic id.

# Axona Minimal, in three steps

Connect — locate the user, derive the identity, build the node + peer:

```
const here      = await whereAmI();           // real GPS, or us-east on denial
const identity  = await deriveIdentity({ lat: here.lat, lng: here.lng });
const transport = webTransport({ bridgeUrl: BRIDGE, identity });
const node      = new NeuronNode({ id: BigInt('0x' + identity.id),
                                   lat: here.lat, lng: here.lng });

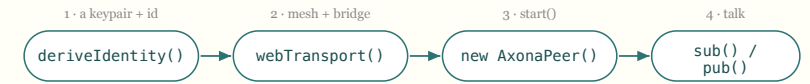
node.transport = transport;
const peer = new AxonaPeer({ domain: new AxonaDomain({ k: 20 }), node, identity, transport });
await transport.start(identity.id);
await peer.start();
```

Subscribe — the handler gets each envelope:

```
await peer.sub(topic, (env) => {
  if (!env || env.deleted || seen.has(env.msgId)) return;
  seen.add(env.msgId);
  const { text, pub } = env.message;           // pub = the node-id we chose to share
  render(text, idLabel(pub), false, topic);   // idLabel → "region:userID"
}, { publisher: ANCHOR.publisher, since: 'all' });
```

Publish — attach our publish id so subscribers can show the region:

```
const msgId = await peer.pub(topic, { text, pub: identity.id },
                               { publisher: ANCHOR.publisher });
seen.add(msgId);
render(text, idLabel(identity.id), true, topic);
```



...which is the whole of **Axona Minimal**:

received  
you · hello, mesh

≈ 60 lines, no framework.  
Open it in two tabs and a  
message crosses between them  
over the live mesh.

[demo.axona.net/apps/axona-minimal](https://demo.axona.net/apps/axona-minimal)

That is the entire networking story. Everything else in the file is DOM glue.

## SHARING THE REGION IS THE APP'S CALL

The protocol signs *who*, never *where*. To show a sender's region this app opts in — it puts its own node-id in the post ( pub ) and reads region:userID off it. The disclosure stays visible, at the app layer.

# Try it, then go deeper

## • Run it.

[demo.axona.net/apps/axona-minimal](https://demo.axona.net/apps/axona-minimal) — open two tabs and watch a message cross between them. Each line shows the sender's region, read off its publish id.

## • Read it.

The Explainer (how the routing works), the Architecture note (kernel · transport · bridge · wire), and the API Reference for every exported symbol.

## • Host it.

Run a relay ( `axona-relay` ) to carry your region's topics, or stand up your own bridge — the one piece you bootstrap from.

### THREE LIVE DEMOS

[demo.axona.net](https://demo.axona.net) — Axona Minimal, the hello-world pub/sub example, and the S2 region globe.

### THE KERNEL

[axona-net/axona-protocol](https://axona-net/axona-protocol)